

Responsible & Trustworthy AI — Learning Resources

KSS × Fusemachines Fellowship · Student Edition

Companion to the module plan *Responsible and Trustworthy AI* (Explainable AI, Causal AI, and the surrounding principles of fairness, robustness, privacy and accountability) and to the runnable code in this repository.

Audience: Fellowship learners and KSS AI / software engineers · **Duration:** 4 sessions (~6 hours) + a 1-hour coding challenge · **Facilitators:** Sujan Sharma / Aashish Pokharel · **Reference book:** *Trustworthy AI*

Learning Objectives (Bloom's Taxonomy)

By the end of this module, all learners will be able to:

1. **Explain the core principles** of responsible and trustworthy AI — fairness, transparency, accountability, robustness/safety, privacy and reliability — and locate them in real deployments governed by the **NIST AI RMF** and the **EU AI Act**. (Bloom's: Remember/Understand)
2. **Apply explainability techniques** (feature importance, SHAP/LIME, partial-dependence plots, counterfactuals) to interpret a pre-trained model's predictions and state the strengths and limits of each. (Bloom's: Apply/Analyze)
3. **Distinguish correlation from causation** using the ladder of causation (association -> intervention -> counterfactual), read a Directed Acyclic Graph (DAG), and identify where causal reasoning makes a system more trustworthy. (Bloom's: Understand/Analyze)
4. **Implement input and output guardrails** — PII detection/redaction, prompt-injection detection, sensitive-topic classification, and schema plus groundedness validation — and explain why each layer exists. (Bloom's: Apply)
5. **Enforce least-privilege tool permissions and human-in-the-loop control** in an agentic system, and estimate the blast radius of a runaway agent. (Bloom's: Apply/Evaluate)
6. **Instrument a trustworthy pipeline** with tamper-evident audit logging, monitoring, incident review and red-teaming, and prove its behaviour with automated tests. (Bloom's: Apply/Evaluate)

By the end of the module, some learners will be able to (depth / stretch):

7. **Compare multiple explanation methods** (e.g., SHAP vs. counterfactuals) and evaluate them with fidelity/stability metrics or a small human-subject study. (Bloom's: Analyze/Evaluate/Create)
8. **Build and query a simple causal model** (DAG plus estimand, DoWhy-style) for intervention or counterfactual analysis on a dataset. (Bloom's: Apply/Create)
9. **Critique a live system** against the NIST AI RMF and the EU AI Act, assemble an evidence pack, and defend a go/no-go deployment decision with named owners, tolerances and review dates. (Bloom's: Evaluate/Create)

Prerequisites: basic supervised machine learning (scikit-learn), Python (pandas, matplotlib/seaborn, Jupyter), and introductory statistics (correlation, p-values). The teaching code in this repository needs **no third-party packages** — it is pure standard-library Python, so every demo runs immediately after checkout.

The running case — Northline Bank "CreditBoost"

All four sessions work on **one composite case** so you walk a whole lifecycle — **development -> deployment -> end-user serving** — instead of four disconnected exercises.

Northline Bank is a mid-size retail bank (1.4M cardholders) operating in the EU and the UK, in scope for the **EU AI Act**, **GDPR** (Art. 15 and 22) and the national supervisor's model-risk guidance. Its product, *CreditBoost*, has two components:

- **The decision model** — a nightly gradient-boosted tree (XGBoost, 240 features, AUC 0.81, retrained monthly) that **raises credit limits by one band** when the expected value clears a threshold, **~40,000 limits per month**.
- **The explanation layer** — an LLM that writes the customer letters and refusals.

The case carries an unexplained **1.7× refusal disparity**, a flawed pilot, two incidents in a quarter, and no post-market monitoring plan. The pitch you have to attack: *"94% of predictions were correct, we publish a model card, every decision is logged, and we passed the EU AI Act conformity assessment — so CreditBoost is trustworthy."*

The four whole-course questions (C1–C4) and their model answers, rubrics and pitfalls live in [5_responsible_trustworthy_ai/00_core_questions.md](../5_responsible_trustworthy_ai/00_core_questions.md); drill them with `python3 5_responsible_trustworthy_ai/quiz.py`.

How the module runs

Session	Focus	Est. time	Core question	Key artifacts in this repo
1	Foundations of Responsible & Trustworthy AI	60–90 min	C1 — Deconstruct the trustworthiness pitch	00_core_questions.md, 4_best_practices/checklist.md
2	Explainable AI (XAI)	90–120 min	C2 — Three audiences, one truth	trust_safety/eval/counterfactual_bias.py, trust_safety/guardrails/output/groundedness_checker.py
3	Causal AI for Deeper Trust (<i>optional</i>)	90–120 min	C3 — The 2.9% trap	questions/scenario_questions.md, DoWhy tutorials
4	Integration, Auditing & Regulation	60–90 min + project	C4 — The 90-minute pre-deployment review	trust_safety/governance/*, trust_safety/orchestrator/autonomy_promotion.py
Eng.	Engineering track — Prompt & Agentic AI safety	hands-on	drills K1–K16 (archive)	1_llm_basics/, 2_agentic_ai/, 3_testing/, trust_safety/

Session 3 is the *optional* session: if causation was already covered in an earlier class, run C3 as a short reading and keep the artifacts for C4.

Session 1 — Foundations of Responsible & Trustworthy AI (60–90 min)

Focus: the principles, the vocabulary, and the two frameworks every trustworthiness claim is judged against.

Core concepts

- **Core principles of trustworthy AI:** fairness (bias detection and mitigation), transparency, accountability, robustness/safety, privacy (e.g., differential privacy) and reliability.
- **The trustworthiness pitch and its gaps:** accuracy is not trustworthiness; a model card is not accountability; logging is not oversight; conformity is a *floor*, not proof.
- **Risk management, not box-ticking:** the **NIST AI RMF** functions — **GOVERN -> MAP -> MEASURE -> MANAGE** — and how a measured-but-unowned disparity is a GOVERN failure.
- **Regulatory frame:** the **EU AI Act** risk tiers and Annex III high-risk categories, plus **GDPR** Art. 15 (access), 22 (automated decisions), 33 (breach) and 35 (DPIA).

- **The lifecycle view:** *development*, *deployment* and *end-user serving* — and which decisions have to be made in each.

From the code in this repo

- `5_responsible_trustworthy_ai/00_core_questions.md` — the running case, C1 and the four-question set.
- `4_best_practices/checklist.md` — the “? never / ? always” checklist and the *usual* vs. *trustworthy* quick-reference table.
- `trust_safety/guardrails/data_classifier.py` and `trust_safety/policies/defaults.py` — data classification and the ComplianceProfile / GDPR_PROFILE compliance profiles.

Try it

```
python3 5_responsible_trustworthy_ai/quiz.py --id C1          # C1 with its model answer
python3 5_responsible_trustworthy_ai/build_questions.py --check # validate the question bank
```

References: NIST AI RMF 1.0 · EU AI Act (Annex III, Ch. III, Arts. 72–73, 86) · GDPR Arts. 15/22/33/35 · *Trustworthy AI* (module book).

Session 2 — Explainable AI (XAI) (90–120 min)

Focus: making a black-box model's behaviour legible to the three audiences that matter — the affected person, the operator and the regulator.

Core concepts

- **Black-box vs. interpretable models:** post-hoc methods (**LIME**, **SHAP**, partial-dependence plots, counterfactuals) vs. intrinsic interpretability (decision trees, rule lists, monotonic/generalised additive models).
- **Global vs. local explanations:** feature importance and PDP summarise the whole model; SHAP and counterfactuals explain one decision.
- **What a good explanation needs:** *fidelity* (does it reflect the model?), *stability* (does it survive small input changes?), and *human understandability*.
- **Three audiences, one truth:** a clinician, a customer and an auditor need different explanations of the same decision — and each carries a different risk of misuse.
- **Explanation misuse:** SHAP/LIME explain the *model*, not the world; a small attribution is not safety; a plausible explanation can *increase* over-trust instead of improving a decision.

From the code in this repo

- `trust_safety/eval/counterfactual_bias.py` — generates prompt pairs that vary only a demographic attribute and measures output divergence (sentiment, refusal rate, length, recommendation quality). This is bias *measured statistically*, not as a single pass/fail.
- `trust_safety/guardrails/output/groundedness_checker.py` — checks that a generated explanation stays grounded in its sources, so the LLM's rationale is not invented.
- `5_responsible_trustworthy_ai/00_core_questions.md` — C2 (“Three audiences, one truth”) with its three-audience table and the SHAP-vs-causal traps.

Try it

```
python3 5_responsible_trustworthy_ai/quiz.py --id C2 --phase serving
```

References: Christoph Molnar — *Interpretable Machine Learning* (core reference) · SHAP and LIME documentation · *XSTest* (over-refusal benchmarking).

Session 3 — Causal AI for Deeper Trust (90–120 min) [optional]

Focus: moving from *what correlates* to *what would change if we intervened* — the question deployment decisions actually turn on.

Core concepts

- **The ladder of causation:** association (seeing) -> intervention (doing) -> counterfactual (imagining).
- **Structural Causal Models (SCMs) and DAGs:** drawing the assumed data-generating process, and reading **confounding**, **colliders** and **mediation** off the graph.
- **Identification:** the *estimand* vs. the estimate; ITT vs. TOT; when a randomised holdout or a regression discontinuity at an existing cutoff buys causal precision worth its cost.
- **Failure modes:** confounding is the default explanation for any observational association; a causal claim without a stated design is an opinion.
- **Why it matters here:** causal explanations are more reliable and more actionable than purely correlational ones — “*the follow-up programme helps*” is a causal claim, not an accuracy metric.

From the code in this repo

- `questions/scenario_questions.md` — Scenario 1, Q5: establish whether enrolling a patient *improves* their outcome rather than merely selecting patients who would have recovered anyway.
- `5_responsible_trustworthy_ai/00_core_questions.md` — C3 (“The 2.9% trap”): the pilot DAG, the estimand, and the decision gate.
- `trust_safety/eval/counterfactual_bias.py` — the counterfactual *reasoning* pattern applied to fairness testing.

Try it

```
python3 5_responsible_trustworthy_ai/quiz.py --id C3 --phase development
python3 5_responsible_trustworthy_ai/quiz.py --id C3 --phase deployment
```

References: *Causal Inference for the Brave and True* (Matheus Facure) · DoWhy / EconML tutorials · Pearl's *ladder of causation*.

Session 4 — Integration, Auditing & Regulation (60–90 min + project work)

Focus: pulling the previous sessions into one defensible pre-deployment review and an evidence pack an auditor can read.

Core concepts

- **Accountability as a mechanism, not a feeling:** a named human-oversight role with authority to override and stop.
- **The evidence pack:** model card, DPIA, Art. 9–15 documentation, Art. 27 fundamental-rights assessment, Art. 72/73 post-market monitoring and incident reports, Art. 86 right to explanation — across **both components** and **both roles** (provider + deployer).
- **Post-market monitoring:** every signal has a **threshold**, **an owner** and **an action** — including group-level refusals and the letter-generation gate.
- **Incident response and the incident loop:** severity, a clock, and feeding resolved incidents back into the test suite as new regression cases.

- **Go / no-go and residual-risk acceptance:** escalate the *risk tolerance* to a human for signature — not the technology choice.
- **Decommissioning triggers** decided *before* a crisis, not during one.

From the code in this repo

- `trust_safety/governance/audit_log/` — append-only, tamper-evident audit store and models.
- `trust_safety/governance/incident_review.py` — the incident-review loop (detection -> investigation -> fix -> regression test).
- `trust_safety/governance/policy_versioning.py` and `policy_docs/constitution.md` — versioned policy with a constitution document.
- `trust_safety/orchestrator/autonomy_promotion.py` — the formal in-loop -> on-loop -> out-of-loop promotion process, gated on a documented safety case.
- `trust_safety/governance/dashboard.py` + `ui/pages/01_Dashboard.py` — the monitoring dashboard.
- `5_responsible_trustworthy_ai/00_core_questions.md` — C4 ("The 90-minute pre-deployment review"), the capstone.

Try it

```
python3 5_responsible_trustworthy_ai/quiz.py --id C4 --type hands_on
python3 -m pytest trust_safety/tests/test_incident_review.py trust_safety/tests/test_audit_log.py -q
```

References: NIST AI RMF (GOVERN/MAP/MEASURE/MANAGE) · EU AI Act Arts. 9–15, 27, 72–73, 86 · *Trustworthy AI*.

Engineering Track — Prompt & Agentic AI Safety (hands-on)

Focus: the engineering counterpart of the four sessions — the concrete guardrails, permissions and governance you build into LLM and agentic systems. This is the part of the module aimed directly at AI / software engineers.

Core concepts (the "For Internal KSS" list)

1. Preventing **prompt injection** — using existing libraries and hand-built guardrails.
2. **Vulnerability identification** in LLM applications.
3. **Data privacy** — what to provide and what not to provide for governance.
4. What **PII** is and how to handle it.
5. Handling **sensitive information**.
6. **Input/output validation** for AI and LLMs.
7. **Biasedness detection**.
8. **Alignment** issues (see Anthropic's alignment work).
9. Handling **safe-request refusal**.
10. **Hallucinations** and how to detect them.
11. **Alignment and autonomy control** — human in / on / out of the loop.
12. **Governance**.
13. **Data poisoning**.
14. **Incident management** — if losses occur, who bears the cost?

Each lab below maps those concepts to runnable code. Run every demo from the repository root.

Lab A — Prompt engineering & input validation ->

1_llm_basics/01_prompt_engineering.py

- **Concepts:** the unsafe `f"Answer: {user_input}"` anti-pattern vs. a defensive pipeline; instruction/data separation with delimiters; structured output contracts.
- **Build with:** `trust_safety.lessons.safety — ContentFilter.sanitize_pii, ContentFilter.contains_sensitive_topic, PromptInjectionDetector.check, PromptInjectionDetector.compute_risk_score, OutputValidator`.
- **Run:** `python3 1_llm_basics/01_prompt_engineering.py`

Lab B — Data privacy & RAG governance -> 1_llm_basics/02_data_privacy_rag.py

- **Concepts:** PII scanning before embedding, per-document access control, data lineage, right-to-forget, and auditing every retrieval.
- **Build with:** `TrustworthyRAG / Document` vs. the `UnsafeRAG` anti-pattern; `AuditLogger, compute_hash`; production PII via `trust_safety/guardrails/input/pii_detector.py` (Presidio-backed, `redact/mask/tokenize`).
- **Run:** `python3 1_llm_basics/02_data_privacy_rag.py`

Lab C — Agentic safety: naive vs. trustworthy -> 2_agentic_ai/

- **Concepts:** tool risk classification, role-based permissions, argument validation, path-traversal protection, human-in-the-loop approval, rate limiting, sandboxing, session boundaries, prompt-injection checks on every tool call.
- **Build with:** `2_agentic_ai/01_naive_agent.py` (the anti-pattern — `shell=True`, unrestricted file access, no approval) vs. `2_agentic_ai/02_trustworthy_agent.py` (`ToolRiskLevel.SAFE...CRITICAL`, `ToolCapability`, `PermissionManager`). Production equivalents live in `trust_safety/orchestrator/` (`tool_registry`, `policy_gate`, `approval_queue`, `circuit_breaker`).
- **Run:** `python3 2_agentic_ai/01_naive_agent.py` then `python3 2_agentic_ai/02_trustworthy_agent.py`

Lab D — Testing & the coding challenge -> 3_testing/

- **Concepts:** unit tests for filters, injection detection and output validation; property-based tests (idempotency, invariants); integration tests for permission enforcement; load tests for rate limiting.
- **Challenge:** build `safe_code_review(code_snippet, reviewer_role)` that passes all 5 cases — input validation, role permissions (`viewer/developer/admin`), output validation, audit trail. See `3_testing/CHALLENGE.md`.
- **Run:** `python3 3_testing/01_llm_testing.py` then open `3_testing/CHALLENGE.md`

Lab E — Best-practice checklist -> 4_best_practices/checklist.md

- **Concepts:** the seven areas — prompt engineering, data privacy & RAG, agentic safety, testing, monitoring & observability, deployment, and architecture decision records — as an auditable checklist.

How to use these questions

The module is built around **"Try First, Understand Later"** and **low floor / high ceiling / wide walls**. C1 can be attempted cold, before any vocabulary, and still produce useful disagreement in the room.

Type	When	Behaviour
in_class	during the plenary (C1–C3)	answered together

Type	When	Behaviour
quiz	quick checks	short, marks per session
discussion	peer debugging	share where an explanation surprised or misled you
essay	reflect	written, marked against the fundamental problem
hands_on	workshop / assessment	C4, the capstone

A wider set of open prompts is available in `questions/session_questions.md`, `questions/agentive_llm_questions.md`, and `questions/scenario_questions.md` (two full scenarios — a hospital ML system and an internal agentive assistant).

Assessment — tests for chapter-level knowledge/skills

- **Quiz** on the core principles and the difference between **XAI** and **Causal** reasoning.
- **Interpret a provided SHAP plot or causal graph** and explain what it does and does not license you to conclude.
- **Short essay**: *"How would you explain this model decision to a regulator?"*
- **Capstone (C4)**: assemble the evidence pack and defend a go/no-go decision with conditions, owners and review dates.

Assignment for the week — "Try First, Understand Later"

Attempt to explain a black-box model (e.g., a trained XGBoost or neural net on a tabular dataset such as **UCI Adult Income** or **COMPAS**) using **basic feature importance** or **partial-dependence plots** *before* the formal XAI/Causal material. Then reflect, in writing, on the limitations of what you produced — what it convinced you of, and what it quietly left out.

Guardrails for the assignment

- Common pitfalls to watch for: SHAP assumptions, and confounding in causal models.
- Ask when to prefer intrinsic vs. post-hoc explanations.
- Peer-debugging prompt: *"Share a case where your explanation surprised you or seemed misleading — why?"*

Main takeaway

Responsible AI is not just about accuracy — it is about building systems that are **transparent, causally grounded, fair, and accountable**, so that humans can trust and effectively oversee them. XAI and Causal AI are the practical enablers: they turn "the model is 94% accurate" into decisions a person can inspect, contest and own.

***The golden rule:** never trust LLM output implicitly — always validate and sanitize, and design for **defense in depth** so that if one layer fails, another catches it.*

References & further reading

- **NIST AI Risk Management Framework (AI RMF 1.0)** — GOVERN / MAP / MEASURE / MANAGE.
- **EU AI Act** — Annex III high-risk categories, Chapter III obligations, Arts. 27, 72–73, 86.
- **GDPR** — Arts. 15, 22, 33, 35.

- Christoph Molnar — *Interpretable Machine Learning* (free online).
- *Causal Inference for the Brave and True* (Matheus Facure) · DoWhy / EconML tutorials.
- **OWASP Top 10 for LLM Applications** (prompt injection, excessive agency, sensitive-information disclosure).
- Anthropic alignment material (RLHF, RLAIIF, Constitutional AI, Responsible Scaling).
- *Trustworthy AI* (module reference book).

Repository map (where the code lives)

```
kss_trustworthy_ai/
|-- 1_llm_basics/           Prompt engineering + data-privacy/RAG demos
|-- 2_agentic_ai/          Naive (anti-pattern) vs. trustworthy agent
|-- 3_testing/             Test suite + the 1-hour coding challenge
|-- 4_best_practices/checklist.md The auditable best-practice checklist
|-- 5_responsible_trustworthy_ai/ Core questions C1-C4, running case, quiz.py
|-- questions/            Session, agentic and scenario question sets
|-- trust_safety/
|   |-- lessons/          Stdlib-only teaching layer (safety.py, config.py)
|   |-- guardrails/       Production input/output guardrails
|   |-- orchestrator/     Tool registry, policy gate, approval queue, autonomy
|   |-- governance/       Tamper-evident audit log, incidents, dashboard
|   |-- llm/ · gateway/ · eval/ · redteam/ · policies/
|   |-- tests/            Test suite for the production stack
|-- ui/                   Streamlit dashboard, approval queue, consoles
```

Generated as a companion document to the KSS × Fusemachines Fellowship module "Responsible and Trustworthy AI". Source of truth: this Markdown file; the PDF is produced from it by `build_pdf.py`.